IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 5 October 2024, accepted 6 November 2024, date of publication 12 November 2024, date of current version 13 January 2025.

Digital Object Identifier 10.1109/ACCESS.2024.3496823

SURVEY

The Impact of YOLO Algorithms Within Fall Detection Application: A Review

AHLAM R. KHEKAN[1,2], HADI S. AGHDASI[1], AND PEDRAM SALEHPOUR[1], (Member, IEEE)
[1]Faculty of Electrical and Computer Engineering, University of Tabriz, Tabriz 51666-16471, Iran
[2]BioMedical Informatics College, University of Information Technology and Communications, Baghdad 10016, Iraq
Corresponding author: Hadi S. Aghdasi (aghdasi@tabrizu.ac.ir)

ABSTRACT The phenomenon of human falls is a highly significant health concern, particularly for elderly individuals and disabled individuals who reside alone. The global elderly population is steadily growing. Consequently, human fall detection is emerging as a highly efficient method for enhancing the quality of life for individuals in need of assistance. Deep learning (DL) and computer vision have been extensively employed for assistive living purposes. This review article focuses on advanced DL model known as YOLO algorithm for non-intrusive fall detection using vision-based methods. Furthermore, we describe the evaluation architecture of YOLO algorithms (YOLOv1 - YOLOv8). Additionally, we provide a survey on benchmark datasets for fall detection. To enhance comprehension, we will provide a concise overview of various metrics employed to assess the efficacy of fall detection systems. This article also provides insight into future advancements in YOLO algorithms for detecting human falls.

INDEX TERMS Computer vision, deep learning, object detection, YOLO, real-time detection, dataset.

I. INTRODUCTION

Fall detection for the elderly people is a critical area of computer vision due to the significant health risks associated with falls in this population [1]. Falls can cause severe injuries, loss of independence, reduced quality of life, and in some cases can lead to mortality [2]. The elderly people are particularly susceptible due to age-related factors, such as decreased muscle strength, balance impairments, and chronic conditions. Timely intervention after a fall is crucial to minimize resulting complications, and fall detection systems offer a promising solution by providing immediate alerts to caregivers or emergency services. Consequently, developing accurate and reliable fall detection technologies is quite imperative for improving the safety and well-being of the elderly population. Furthermore, with the increasing global population, the incidence of falls is on the rising, highlighting the urgent need for effective prevention and intervention strategies.

Artificial intelligence (AI) has become as a promising tool for addressing the above challenges [3]. By leveraging advanced algorithms and machine learning models, AI-powered fall detection systems can accurately identify fall events, trigger immediate alerts, and provide valuable insights for preventing future falls. These systems and models can analyze various data sources, including video footage, wearable sensor data, and environmental information, to detect fall patterns and assess fall risk factors. Ultimately, the integration of AI into fall detection scenarios holds the potential to significantly improve the safety and well-being of elderly individuals, empowering them to live independently with greater confidence and peace of mind. Among all of the AI models, DL has emerged as a powerful tool for addressing the challenges of fall detection [4]. The YOLO algorithm [5], renowned for its real-time object detection capabilities, has shown huge potential in this research field. By training YOLO models on extensive datasets of human actions and interactions, including falls, it is possible to achieve high accuracy in detecting fall events from video or image sequences. Compared to traditional methods, this technology can provide a number of improved advantages, including faster processing speed, the ability to manege complex background scenarios, and the potential to identify various fall patterns. Integrating YOLO-based fall

The associate editor coordinating the review of this manuscript and approving it for publication was Chuan Li.

© 2024 The Authors. This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

detection systems into smart homes and or wearable devices, can actually revolutionize elderly care by providing proactive monitoring and timely assistance, ultimately enhancing their safety and independence.

In light of the importance of noise-free images for accurate diagnosis, researchers have shifted their attention from improving algorithms to focusing on image enhancement, as described above. Machine learning was once in high demand [6], but today, practically all eyes are on deep learning. It is safe to say that the conventional wisdom regarding the arcane training process of deep learning has been radically revised in light of fine-tuning and transfer learning. Researchers are currently using deep learning models combined with data augmentation to improve the efficiency and generalizability of deep learning models. In recent years, researchers have turned their attention to hybrid models capable of multi-task learning, where the main objective of these multi-task learning approach is to reduce the complexity and increase the efficiency of fully autonomous systems by teaching models to learn many functions from a single input. Previous research and studies cover the YOLOs algorithm and its use in object detection operations. It reviews YOLO algorithms based on basic architectures, comparative and incremental methods in this category, common datasets and the results obtained accordingly. Existing literature highlights the importance of using fall detection systems in preserving the lives of older adults. As these techniques enable the opportunity for rapid rescue intervention, this will reduce the physical and mental consequences resulting from a fall [7]. However, the revolution of YOLO versions impact the object detection, especially in fall detection. Therefore, this review focus on the structure development of YOLO algorithms, and their association in fall detection. Furthermore, we review existing benchmark dataset, and evaluation metrics. The major important contributions of this paper can be summarized as follows:

- Review the most related fall procedure in the field of Artificial Intelligent (AI) networks
- Describe all related YOLO versions with their structure.
- Describe all related metrics, and datasets in the field of fall detection.
- We extend our discussion of the fall detection with YOLO models with in-depth and fresh prospective.
- We provide new research direction for the reader to further explore the field of fall detection with YOLO model.

The rest of this paper is organized as follows: In Sec. II, we provide a major outline of the fall detection procedure. Sec. III is devoted to describe all YOLO version network structure. In Sec. IV, we describe the evaluated metrics. In Sec. V we discuss and compare fall datasets. Sec. VI provides in-depth discussion on fall detection with YOLO models. In Sec. VII, we explore the potential future directions of the major limitations for the related work, and finally we conclude our work Sec. VIII.

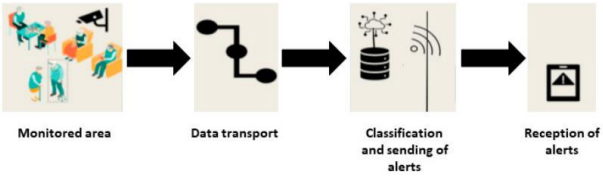


FIGURE 1. Components for a fall detection model.

II. FALL DETECTION PROCEDURE

Fall detection has seen significant advancements through the application of AI models [8]. By employing various sensor modalities and equipment, such as wearable devices, cameras, and pressure sensors, these systems collect data on human movement. Advanced machine learning algorithms, including support vector machines [9], random forests [10], and deep neural networks [11], are trained on extensive datasets to differentiate between falls and daily living activities. This enables real-time monitoring and rapid response to fall incidents, providing essential safety and peace of mind for individuals at risk. In this section, we will review the main approaches procedure which handle fall detection with learning models.

A. MACHINE LEARNING MODELS

Due to the huge impact of AI models in image processing techniques in different fall scenarios, including healthcare, sports, and elderly care, researches tens to deploy Machine learning models such Support Vector Machine (SVM) [12], K-nearest neighbors (KNN) [13], and Random Forest [14] as a promising tool in fall detection. In addition, many researchers studied detecting fall events using a video surveillance camera [15]. For example, Anishchenko et al. [16] presents a method for fall detection with a bio-radar system.

1) SUPPORT VECTOR MACHINE

One of the ML techniques that is utilized the most frequently, is SVM, which is actually a supervised learning models. These models are accompanied by associated learning algorithms that analyze data for the purposes of classification and regression analysis. SVMs are capable of performing nonlinear classification in an efficient manner with a kernel trick, which implicitly maps the data inputs into high-dimensional feature spaces. This learning process can provide to the SVMs with the ability to perform linear classification as well. In essence, these techniques can establish a boundaries between different classes. The margins are drawn in such a way that the distance between the margin and the classes is maximized, which in turn reduces the amount of classification error that occurs [17].

Charfi et al. [18] proposed model based SVM classifier for fall detection within home environment. By using this classifier, the input layers was fed by a set of selected features detected and extracted from human body silhouette tracking.

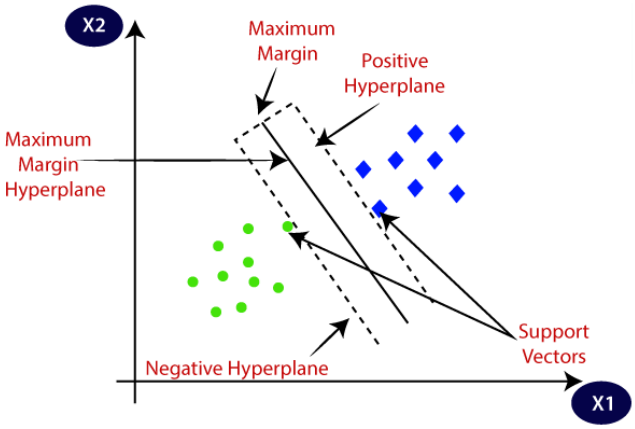


FIGURE 2. Support-vector-machine-algorithm.

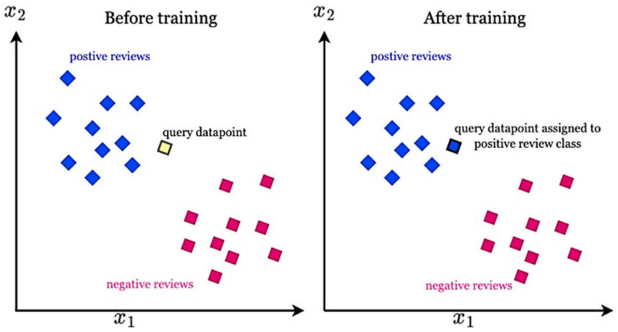


FIGURE 3. K-nearest neighbors algorithm.

The results showed that with accurate minimum learning loss, the model can achieve high performance using a single camera in the scene.

2) K-NEAREST NEIGHBORS

The k-Nearest Neighbors (kNN) algorithm is a straightforward yet potent ML technique, applicable to both classification and regression tasks. While effective in both domains, it is particularly renowned for its classification applications, by identifying clusters of similar data points, kNN assigns new, unlabeled data to the category that most closely resembles its nearest neighbors, making it a valuable tool for predictive modeling. As demonstrated by Taunk et al. [19], kNN can be effectively employed for practical applications such as fall detection. By utilizing off-the-shelf devices and the kNN classification algorithm, Liu et al. [20] proposed a system capable of accurately detecting fall incidents based on human body silhouette analysis. Their experiments revealed a promising detection rate with 84.44% accuracy, highlighting the algorithm's potential for real-world applications

3) RANDOM FOREST

The Random Forest algorithm, was introduced by Biau and Scornet [21], has emerged as a versatile and powerful tool for both classification and regression tasks. By aggregating

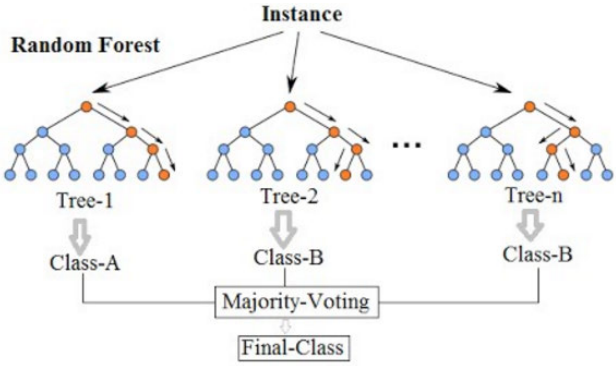


FIGURE 4. The random forest algorithm.

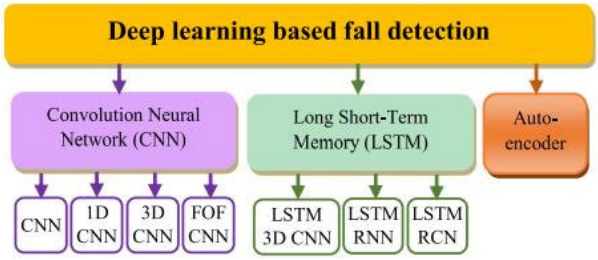


FIGURE 5. Deep learning methods used for fall detection.

the predictions of multiple randomized decision trees, this ensemble pipeline will excels in scenarios with high-dimensional data. Random forest ability to handle large-scale problems, adaptability to various learning objectives, and provision of variable importance measures make it a valuable asset in the ML applications. In a recent study, Liu et al. [22] proposed a hybrid intrusion detection model that leverages machine learning and deep learning techniques. Employing both k-means and Random Forest algorithms for binary classification, the model efficiently categorizes normal and attack events on the Spark platform. To further refine and improve the classification of abnormal events, Convolutional Neural Networks (CNNs), Long Short-Term Memory (LSTMs), and other deep learning algorithms were integrated. To address the imbalance and inconsistency in the dataset, the ADASYN technique was utilized. The performance of the proposed model was evaluated using the NSL-KDD and CIS-IDS2017 datasets. Experimental results demonstrate a superior true positive rates for most fall events, faster data preprocessing, and reduced training time. Notably, the model achieved accuracy with multi-target classification of up to 85.24% on the NSL-KDD dataset and 99.91% on the CIC-IDS2017 dataset.

B. DEEP LEARNING MODELS

1) CONVENTIONAL NETWORK

Neural networks, particularly CNNs, have emerged as a powerful tool for fall detection applications, leveraging their proficiency in analyzing sequential and visual data.

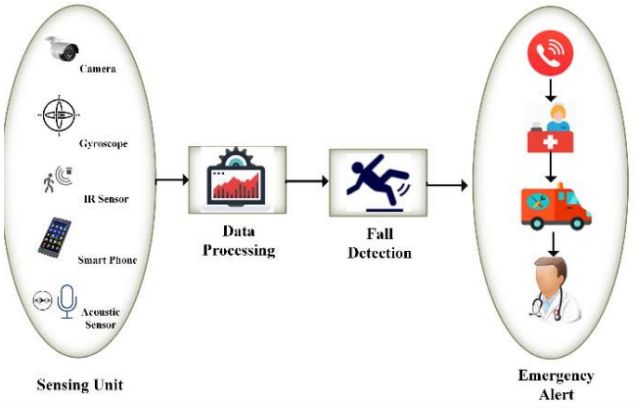


FIGURE 6. Fall detection in many techniques.

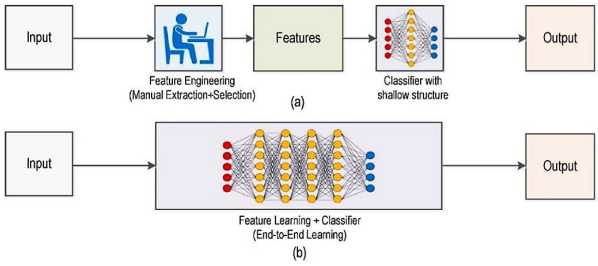


FIGURE 7. (a) Traditional computer vision workflow vs. (b) Deep learning workflow [23].

CNNs have demonstrated effectiveness in tasks such as image classification, natural language processing, and human activity recognition. Santos et al. [24] proposed the use of CNN-based models for fall detection using accelerometer data, while Yu et al. [25] adapted CNN architectures for human activity recognition and subsequently extended them to fall detection. Based on the above studies, recent research by Yadav et al. [26] introduced latent feature pooling within CNN models for enhanced fall detection in activities of daily living (ADLs). Additionally, long short-term memory (LSTM) networks have proven to be effective in distinguishing between falls and ADL events, as illustrated in Fig. 7.

2) YOLO MODEL

When discussing deep learning with the YOLO (You Only Look Once) algorithm, it's important to highlight the significant impact YOLO has had on object detection tasks within the field of computer vision. Here are some key points [27] to consider when discussing deep learning with YOLO:

- Object Detection Efficiency: YOLO is known for its efficiency in object detection tasks. Unlike traditional methods that scan an image multiple times at different scales, YOLO divides the image into a grid and predicts bounding boxes and class probabilities directly. This single forward-pass approach enables real-time object detection.
- Real-Time Performance: One of the major strengths of YOLO is its ability to achieve real-time performance. By processing the entire image in one pass, YOLO can detect objects quickly and accurately, making it suitable for applications requiring fast inference speeds, such as autonomous driving, surveillance, and robotics.
- Single Shot Detector: YOLO follows the single shot detection model (SSD), where object detection and classification are performed simultaneously in a single network. This design simplifies the detection pipeline and leads to faster inference times compared to two-stage detectors like Faster R-CNN.
- Architectural Innovations: YOLO has undergone several iterations, with YOLOv3 and YOLOv4 being some of the most popular versions. These iterations introduced architectural innovations such as feature pyramid networks (FPN), darknet backbones, and advanced post-processing techniques to improve detection accuracy and speed.
- Applications in Various Domains: YOLO has been widely adopted across various domains, including object detection in images and videos, pedestrian detection, face detection, and more. Its versatility and speed make it a preferred choice for researchers and practitioners working on real-time computer vision tasks.
- Challenges and Limitations: While YOLO excels in speed and efficiency, it may struggle with detecting small objects or objects in crowded scenes. Handling scale variation and precise localization of objects can be challenging for YOLO architectures, prompting researchers to explore solutions to improve performance in these scenarios.
- Future Directions: Ongoing research in the field of deep learning with YOLO focuses on improving the accuracy and robustness of object detection, exploring multi-scale features, incorporating attention mechanisms, and enhancing the model's interpretability and generalization capabilities.

Based on the above YOLO features, we will focus our proposal in this review on YOLO models, with their evolution of fall detection.

III. YOLO: YOU ONLY LOOK ONCE

The algorithm model known as YOLO is a breakthrough in computer vision applications, especially in the field of real-time object detection. This model was introduced in 2016 by Redmon et al. [5], YOLO stands out for its speed and efficiency, making it a valuable tool for various computer vision applications. This introductory section dive into the core concepts of YOLO, highlighting its cons and pros, with respect to traditional object detection methods.

Traditional object detection approaches often depends on a two-stage process. The first stage, a utilization of sliding window technique to scan the image for potential objects. As for the second stage, they classify these potential objects and refine their locations within the image. Although these

techniques are effective, this two-stage approach can be computationally expensive, and hindering real-time applications. On the other hand, YOLO takes a different approach, operating as a single-stage detector [28]. It leverages a single convolutional neural network (CNN) to process and analyze the entire image in one pass. The CNN model divides the image into a grid of cells, then predicts bounding boxes and class probabilities for objects within each cell. This single-stage design simplifies the object detection process and significantly reduces processing time, making YOLO model ideal for real-time object detection tasks.

The success of YOLO has create numerous iterations, each improving upon the previous versions in terms of accuracy, speed, and robustness. These advancements position YOLO as a leading algorithm in the field of object detection, with applications in autonomous vehicles [29], security systems [30], and various image and video analysis tasks [31].

A. NETWORK STRUCTURE

The YOLO algorithm consists of three primary components: the backbone, neck, and head, as illustrated in Fig. 8. The backbone serves as the foundation of the network, tasked with extracting informative features from the input image. A CNN, typically pre-trained on large-scale datasets, is commonly employed for this purpose. Popular backbones in YOLO include VGG16 VGG16 [32], ResNet50 [33], CSPDarknet53 [34], and EfficientRep [35].

The neck acts as an intermediary between the backbone and the head. It comprises two key modules: spatial pyramid pooling (SPP) and path aggregation network (PAN). The neck's role is to integrate feature maps from different layers of the backbone and forward them to the head. Common neck architectures in YOLO include [36], Feature Pyramid Network (FPN) [37], NAS-FPN [38], and Rep-PAN [35].

The head is responsible for processing the combined features and generating the final predictions, including bounding boxes, objectness scores, and classification scores. Popular head options in YOLO include Efficient decoupled [39], Multi-scale [40], and anchor-based detection heads [41]. The choice of backbone, neck, and head significantly impacts the speed and accuracy of the YOLO model and is determined by the specific application and desired trade-offs [42]. Recent iterations of YOLO have incorporated advancements in all three components to enhance overall performance.

B. YOLOv1

The YOLO algorithm is a new step in the field of object detection. The object detection method takes a regression problem that ends up outputting spatially discrete bounding boxes and their associated class probabilities. YOLO is one of the fastest region suggestion methods due to its ability to predict bounding boxes and class probabilities in a single evaluation using a single network. The structure of YOLO is relatively simple. It includes 24 convolutional layers followed by two fully connected layers. The activation

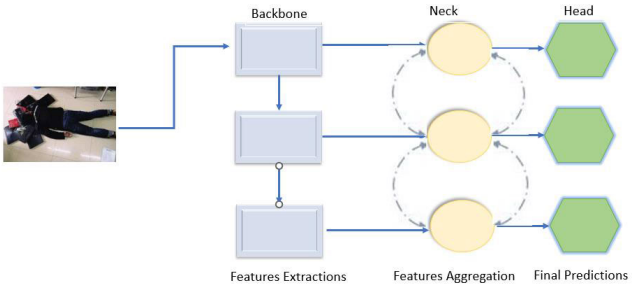


FIGURE 8. General YOLO network structure.

function of the network is Leaky ReLu. Multiple bounding boxes and corresponding class probabilities are predicted simultaneously by a single convolutional neural network. Traditional object detection models often faced speed limitations, requiring multiple passes over the image to identify objects. This approach, known as region-based methods, was computationally intensive and hindered real-time applications. In contrast, YOLO, with its unique approach to one-shot detection, has greatly improved the speed of object detection. By dividing the entire image into a small grid and making predictions directly within each grid cell, YOLO achieves amazing real-time processing capabilities, making it especially useful for applications that require fast and accurate object identification in dynamic environments [43]. If we compare traditional object detection methods with YOLO, we find that it has many advantages. First, it's very fast. The algorithm does not need a complex path because it models detection as a regression problem [5]. Second, it predicts bounding boxes and labels at the same time. It is a significant improvement over traditional two-step methods. Third, YOLO learns globally generalizable representations of objects and causes related to an image. When it comes to unexpected inputs or a completely different field, YOLO is unlikely to fail. YOLO is a state-of-the-art, real-time, end-to-end object detection algorithm that has gained significant attention in the computer vision community, proposed by [5].

YOLO is a single-stage detector as it can be seen by Fig. 9, which means that it can detect all the objects in an image in a single forward pass through a convolutional neural network (CNN). It uses a single neural network to predict the bounding boxes and class probabilities of the objects present in an image [44]. This makes YOLO very fast, and it can achieve real time speeds on even a modest GPU [45]. YOLO divides input image into S x S grid of cells and each of these grids that the center of the object falls into will be responsible to detect the object in the image. In addition, each grid cell predicts B bounding boxes and probabilities of C class. Each bounding box consists of 5 components (x, y, w, h, confidence). The (x, y) are the center of the bounding box and bounded between 0 and 1. Also, (w, h) are dimensions of bounding box and its value fraction of the width and height of the whole image fall between 0 and [46]. Each bounding box forecasts an offset value and a class probability. Bounding boxes are suppressed

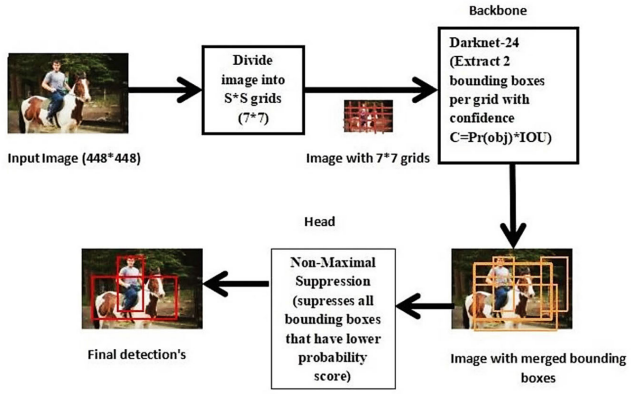


FIGURE 9. Architecture of YOLOv1.

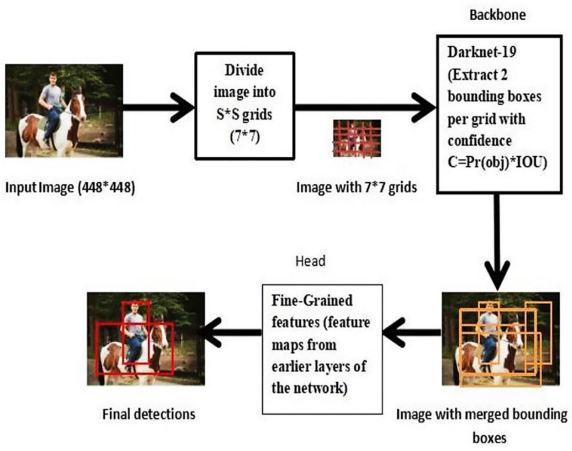


FIGURE 10. Architecture of Yolov2.

that predict class probabilities below a specific threshold, Fig. 9 provides a visual breakdown of the procedure involved in object detection using YOLO version.

The major limitations of this variant are [47]:

- A YOLO detector can only detect one thing per grid, hence the greatest number of objects it can detect always depends on the grid's dimensions. For instance, if the grid size is S × S, then mostly objects can be found is S × S.
- The maximum number of objects YOLO can detect is 1 but when there is more than one object within a grid, it makes an incorrect detection.

C. YOLOv2

An enhanced variant of YOLO, also known as YOLO9000, has been proposed by [5]. It not only outperforms modern models like Fast R-CNN and Faster R-CNN in terms of performance and efficiency but also completes detection in an acceptable length of time. To address the drawbacks of YOLO version 1, the creators of this version of the detector made a number of adjustments to the architecture. Some remarkable architectural modifications which are done in YOLO version 1:

- The modifications of the detector are enhanced and the possibility of overfitting is removed with the addition of this batch normalisation after every convolutional layer, all without the need for dropout layers.
- As opposed to the YOLO version, this enhanced model eliminates using a fully connected layer from the architecture and a fully connected layer above the convolutional layer to predict the offset value, we use anchor boxes to predict the objectness score. Even though YOLO version 2's mAP is lower than YOLO version 1's, the inclusion of anchor boxes raises the latter's Recall value.
- Direct Location Prediction: This attribute mostly relates to the method's stability with the addition of anchor boxes, as anchor boxes to some extent increase the model's instability. The authors used logistic activation to confine the bounding box coordinates inside [0 1] in order to boost the model's stability.
- Fine-grained features: A pass-through layer is added to the network and instead of using features with varying resolutions to execute the network, both low and high-resolution features were viewed horizontally rather than geographically lined up and concatenated.
- Unlike the YOLO detector, which employs images with a dimension of 224 × 224 for training and 448 × 448 for testing. The effectiveness in performance of the YOLO detector version 1 is reduced by the abrupt increase in resolution of image during the testing phase. The network is fine-tuned and trained on images having size 448 × 448 for approx 10 epochs in YOLO v2 in order to overcome this issue. As a result, the issue with the rapid rise in image dimension in YOLO causing a fall in mean Average Precision (mAP) is resolved.
- YOLO version 1 trains the network using bounding boxes that have been manually annotated, but the authors in [5] have trained the network where bounding boxes are produced using the k-means technique in conjunction with their suggested evaluation metric i.e. distance to make learning more straightforward. Distance (box, centroid) = 1-IOU (box, centroid)

The standard value in [5] of "k" in their work to be 5, as this attains a reasonable commutation among the complexity and performance of the network.

D. YOLOv3

An updated version of the YOLO detector created by [48] is called YOLO version 3. Because YOLO version 3 only permits the prediction of one class per item and cannot effectively handle multiclass prediction, it does not employ the softmax classifier to predict the classes of observed objects. To get around this problem, YOLO version 3 uses distinct logistic classifiers for each class [49], and DarkNet-19 uses a hybrid feature extraction strategy and the residual network, in contrast to YOLO v2 which utilizes Darknet-19

as a feature extractor. The YOLO v3 proposed design features a number of alternative connections [50], which improves its efficiency in terms of performance while identifying small items but diminishes it when detecting large and medium objects. Although YOLOv3 was speedier than YOLOv2, it didn't offer any revolutionary improvements over its predecessor. In terms of speed, accuracy, and class specificity, YOLOv3 and earlier versions differ significantly. In terms of mAP and intersection over union (IOU) values, YOLOv3 is quick and accurate. The AP for small objects increased by 13.3 in YOLOv3 is a significant improvement over YOLOv2. Even yet, RetinaNet still outperforms all objects (small, medium, and large) in terms of average precision (AP). There are notable gaps among YOLOv3 and other variants with reference to precision/accuracy, speed and class specificity.

- Precision/Accuracy for tiny/small objects: The AP for tiny or small objects increased by 13.3 in YOLOv3, a significant improvement over YOLOv2. Even yet, RetinaNet still outperforms all objects like small, medium-sized, and large in terms of average precision (AP).
- Speed: While YOLOv3 currently uses Darknet-53 as its backbone for feature extractor [51]. Darknet-53 is more formidable than Darknet-19 and also more efficacious than other backbones as it utilizes 53 convolutional layers as hostile to the precursory 19 layers as in ResNet-152 or ResNet-101. In connection with the values of IOU and mAP, YOLOv3 is expeditious and specific.
- Specificity of Classes: Binary cross-entropy and Independent logistic classifiers are utilized by the new YOLOv3 to predict classes during training. These changes enable the use of complicated datasets for YOLOv3 model training, including Microsoft's Open Images Dataset (OID). For photos in the collection, OID has a large number of overlapping labels, including ''man'' and ''person.'' The YOLOv3 enables classes to be more detailed using multilabel technique and to have several parameters for each bounding box and when a softmax is utilized; each bounding box can only belong to one class.

Object detection and tracking is a technique that aims to recognise individuals within a video and monitor their movements. For YOLO model, Lu and Chu [52] employs YOLOv3 identify the individual, the chair, and the bench. Subsequently, it employs the object tracking method known as continuously adaptive mean shift (Camshift) to continuously track the individual. The steps involved in object detection. First, in his method namely IFADS takes a frame X from a video stream and extracts itself. Furthermore, because the YOLO network reduces the input by a factor of 32, the proposed method ensure that the size of the image, is a multiple of 32. The third step is to load the model parameter of YOLOv3 so that it can identify the individual. The YOLO detector is responsible for detecting the bounding box, which falls under the fourth category. Furthermore, in the case of a box, if it recognisesas the chair or the bench, it assigns

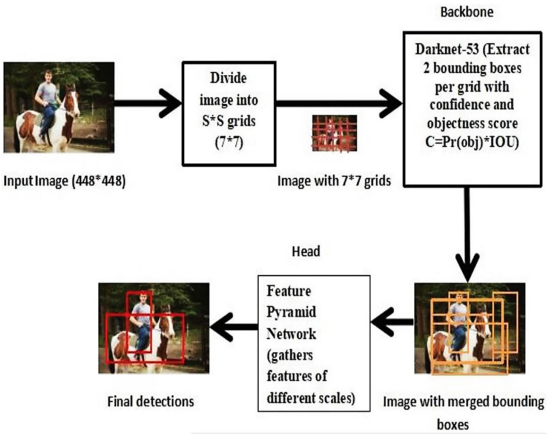


FIGURE 11. Architecture of Yolov3.

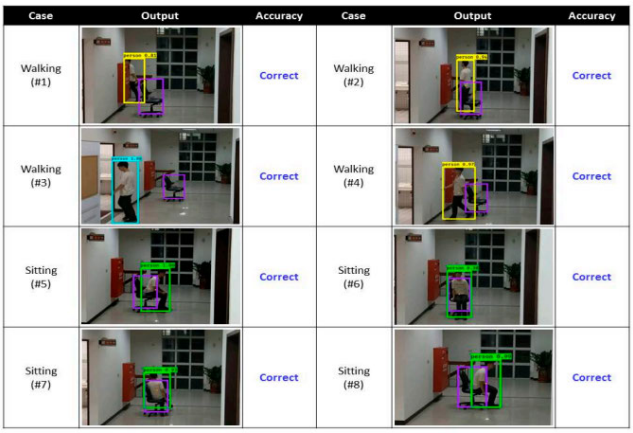


FIGURE 12. The results of a common scenario test.

the word to it. For a bench that does not have a seatback, we consider the height of the bench to be twice as high as the height of a bench that does have a seatback. This is the case if it identifies I as the bench. In contrast, if IFADS identifies as the individual, it will give it the name. When people are moving around, however, they might be obscured by the chair or the bench that is at their disposal. We are able to determine the position of the chair or bench as well as its size because the location of the chair or bench in a public area, such as a park or a courtyard, is consistent. We use a technique known as Camshift, which is an object tracking method, to track people in public areas because there are typically a lot of people there. Fig. 12 represent this scenario.

E. YOLOv4

The YOLOv4 [53] and its design was inspired by a number of Bag-of-Specials and Bag-of-Freebies item recognition techniques. Its accuracy is increased but its inference time and training costs are increased with the Bag -of- Freebies approach and, to a lesser extent, with the Bag-of-Specials method. There were also improvements to the YOLOv4 model in the form of genetic algorithms for selecting the best

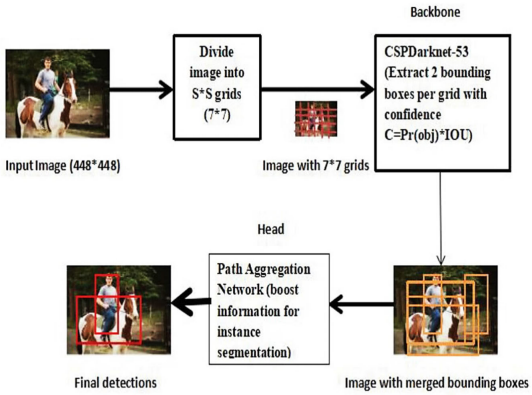


FIGURE 13. Architecture of Yolov4.

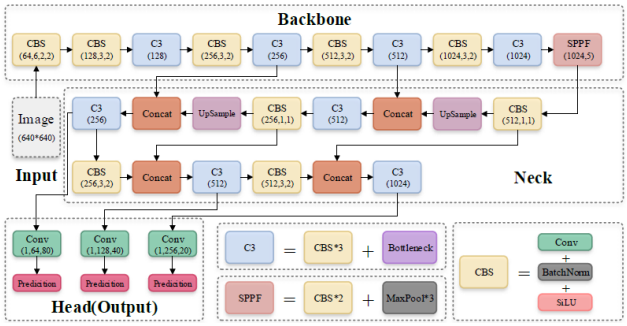


FIGURE 14. Architecture of Yolov5.

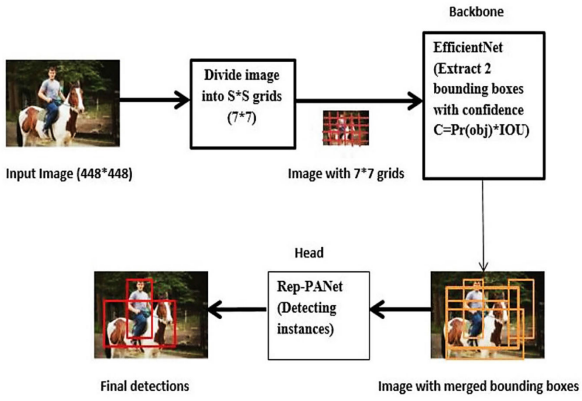


FIGURE 15. Architecture of Yolov6.

hyper-parameter values, SAT (Self-Adversarial Training) and Mosaic techniques for data augmentation, as well as adjustments to existing methods such as Cross mini-Batch Normalization and Spatial Attention Module. The CSPResNext53 and CSPDarknet50 are both built on top of DenseNet. The dense network consists of convolutional neural networks connected together, solving the vanishing gradient problem (back-propagating loss signals through a dense network is difficult), improving feature propagation, promoting reuse of features, and reducing network parameters by reducing the number of layers. CSPResNext50 and CSPDarknet53 have modified DenseNet so that the feature map of the plinth layer is split into two in which one copy is send through the dense block and other one to the next step. EfficientNet performs better in terms of image classification than other networks. However, the authors of YOLO v4 postulate that the networks used as backbone might shows better performance in the models of object detection and choose to analyse them all. The YOLOv4 network uses CSPDarknet53 for the backbone network for experimental results [53].

F. YOLOv5

This is the first version of YOLO [54] to be created in the PyTorch framework, as opposed to earlier versions, which were developed using the Darknet research framework. Due to PyTorch's ease of configuration over Darknet, version 5 of YOLO is now significantly more production-ready than earlier iterations. The run-time of this version of YOLO is another noteworthy advancement. In comparison to its earlier suggested versions, YOLO version 5 is considerably faster. When developed using the same PyTorch library as YOLO 5, YOLO v4's inference time is 50 frames per second while YOLO v5's is 140 frames per second. The YOLO v5 is smaller in size than YOLO v4 and thus is faster. Like other single-stage object detectors, YOLO v5 contains three influential components i.e. Neck, Backbone and Head. The intent of Backbone is to extricate remarkable information from the supplied input image. To extricate salient features from an image given as input to YOLO v5, the Cross Stage Partial Networks (CSP) are employed as its backbone. CSPNet has exhibit a considerable depletion in processing time. The key purpose of Neck is to fabricate feature pyramids. They allow models to successfully scale objects in general. Recognizing the same thing in different scales and sizes is useful. On unobserved data, feature pyramid models perform well. In YOLO v5 model to acquire the feature pyramid, PANet is used as a neck. The Head is used for the final detecting process. It gives final output vectors and the vector comprises of bounding using anchor boxes on the features boxes, objectness scores, and class probabilities. The heads of the YOLO V3 and V4 versions are interchangeable with the head of the YOLO v5.

G. YOLOv6

YOLOv6 [55]accomplishes the best commutation in terms of metrics like speed and accuracy. Modern quantization techniques, such as QAT (quantization-aware training) and PTQ (post-training quantization), are investigated and integrated into YOLOv6 to increase inference speed without significantly degrading performance networks [55]. The main modifications of YOLOv6 are as:

- Different size line networks are redesigned for industrial applications in a variety of contexts. To achieve the greatest speed and accuracy swap, the designs at different sizes differ, with tiny models having a

simple single-path backbone and more models being constructed on effective multi-branch blocks.

- A self-distillation approach is a feature of YOLO v6, and it is used for both classification and regression tasks. To help the learner learn knowledge more effectively across all training phases, the teacher's expertise and labels are dynamically adjusted.
- Substantiate the latest methods of object detection for loss function, label assignment, and data augmentation generally before deciding which ones to use to improve performance.
- RepOptimizer [56] and channel-wise distillation [57] are used to reform the quantization strategy for detection, resulting in an ever-faster and more precise object detector with 43.3% MS COCO AP and a throughput with batch size of 32 is 869 FPS.

H. YOLOv7

Computer vision and machine learning are buzzing about the YOLO v7 [58] model. YOLO's most recent and fast algorithm outperforms other earlier object detection algorithms and YOLO iterations in terms of speed and precision. It can be taught significantly on tiny datasets without any pre-learned weights than other neural networks and requires technology that is several times less expensive. Therefore, YOLOv7 is anticipated to overtake YOLOv4 as the current state-of-the-art for real-time demands and become the caliber for object detection in industries in the future [47]. The authors of YOLOv7 build on prior research on the subject while taking into account the memory requirements for maintaining layers in memory and the distance over which a gradient can propagate back through the layers; the smaller the gradient, the more productively their network will be able to master. They settle on E-ELAN, an extended variant of the ELAN computational block, as their final layer aggregate. The depth, breadth, and resolution of the network that was used to train the network are frequently taken into account by object detection models. In YOLOv7, the authors concatenate layers while scaling the network's depth and width simultaneously. Studies on ablations demonstrate that this method maintains the ideal model design while scaling for various sizes. Re-parameterization techniques build a model that is more resilient to the broad patterns they are trying to capture by averaging a group of model weights. Module-level reparameterization, in which individual network components have their own re-parameterization techniques, has been the recent focus of research. The authors of YOLOv7 experiment with various levels of supervision for this head before deciding on a coarse-to-fine definition in which supervision is handed back from the lead head at varying granularities. Main YOLO network improvement initiatives from V1 to V7. In YOLO, the division into grids is the key point of the detection of objects and confidence loss. In YOLOv2, full convolutional networks, two-stage training, and anchors using K-means are employed. Multi-scale detection with FPN is employed in YOLOV3. SPP,

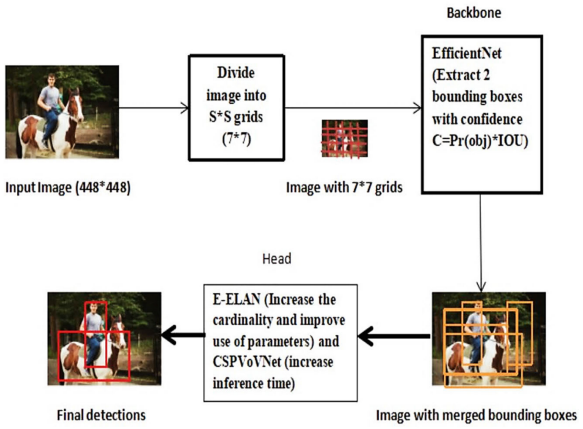


FIGURE 16. Architecture of Yolov7.

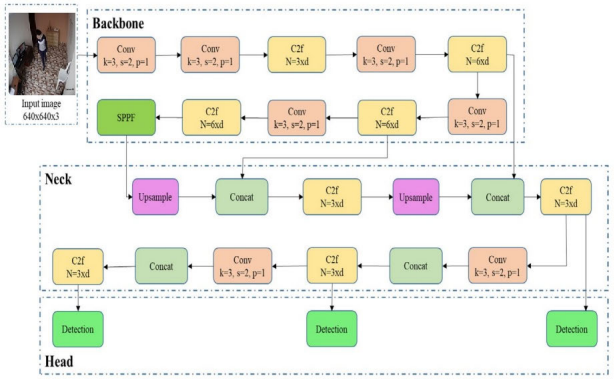


FIGURE 17. The architecture YOLOv8 algorithm.

MISH activation function, and data improvement are utilized in YOLOv4. The YOLOv5 model with its variable model size management utilizes the Hardswish activation function, and data augmentation, as well as Mosaic/Mixup and the GIOU (Generalized Intersection over Union) loss function. YOLO v6 is quantized with QAT and PTQ for better speed and accuracy. The latest version of YOLO i.e. YOLOv7 is settled on EELAN

I. YOLOv8

YOLOv8 is the latest version of the YOLO family of object detection algorithms, which was created in January 2023 by Ultralytics [59], known for its real-time performance and high accuracy. YOLOv8 builds on the strengths of its predecessors while introducing improvements in speed, accuracy, and efficiency. It employs a single neural network to predict bounding boxes and class probabilities directly from full images in one evaluation, making it highly suitable for real-time applications like fall detection [60]. It is the enhanced version of the YOLO algorithm chosen for the proposed system due to the higher accuracy and higher frames per second (fps). various applications have utilized YOLOs for object detection and recognition in various contexts and performed tremendously well. YOLO algorithm has emerged

TABLE 1. Comparison on different YOLOv8 versions [64].

Model	Layers	Input	Head	Parameters	GFLOPS	Blocks
Yolov8S	225	320*320	128	31.5M	28.8	19
Yolov8L	365	640*640	256	43.6M	165.7	19
Yolov8X	365	1280*1280	512	68.8M	258.5	38

as a cornerstone for real-time object detection in various applications such as robotics, autonomous vehicles, and video surveillance [61]. Figure 17 illustrates the overall network structure of YOLOv8, highlighting its key components. It's important to note that YOLOv8 is available in multiple variants (YOLOv8n, YOLOv8s, YOLOv8m, YOLOv8L, and YOLOv8x) [62], catering to diverse computational resources and accuracy needs. Each variant was optimized for different deployment scenarios, from mobile devices with limited processing power, to cloud-based systems demanding high accuracy and throughput [63].

Table 1 presents a comparative analysis of the most popular YOLOv8 versions in the context of fall detection, with respect to the detection accuracy.

IV. EVALUATION METRICS

The results of the system used in prediction, detection and tracking can be verified. There are appropriate evaluation metrics to calculate the accuracy that are used in measuring and verifying the performance of the model [65], [66]. There are cases where this metric cannot be relied upon, i.e. it may not represent the true judgment of the model. Therefore, several other evaluation metrics can be used. Using a single evaluation metric for evaluation is not sufficient to evaluate the model. The model may achieve satisfactory results using a single metric, such as the ''accuracy score'', but it may perform poorly when we evaluate it using other metrics, such as the ''logarithmic loss''. The following subsections describes the most popular metrics used in the fall detection.

A. ACCURACY

Accuracy is a metric that quantifies the overall correctness and validation of a model's predictions, representing the ratio of correct predictions to the total number of predictions [65]. In the context of binary classification, True Negative (TN) refers to instances correctly classified as negative, False Positive (FP) denotes instances incorrectly classified as positive, and True Positive (TP) represent instances correctly classified as positive, where precision and recall for each class can be calculated by:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{1}$$

B. PRECISION

Precision is a metric that measures the proportion of valid correct positive predictions made by a model [65]. It can be is estimated by dividing the number of true positives (instances correctly identified as positive) by the sum of true positives and false positives (instances incorrectly identified as positive) [43]. The resulting precision value falls within the range of 0 to 1, as shown in the following equation

$$Precision = \frac{TP}{TP + FP} \tag{2}$$

C. RECALL

Recall is a metric that measures the proportion of actual positive cases that the model correctly classify. It is obtained by dividing the number of true positives (instances correctly identified as positive) by the sum of true positives and false negatives (instances incorrectly identified as negative) [67]. The resulting recall value falls within the range of 0 to 1, as illustrated in the following equation [65].

$$Recall = \frac{TP}{TP + FN} \tag{3}$$

D. F1 SCORE

The F1-score is a comprehensive metric that evaluates the accuracy of a model with respect to precision and recall metrics. It effectively balances the trade-off between correctly classifying instances and avoiding a significant number of false detection [65]. The F1-score ranges from 0 to 1, with higher values indicating better performance, and can be calculated by the following:

$$F1\ Score = 2 \times \frac{Precision \times Recall}{Precision + Recall} \tag{4}$$

E. MEAN AVERAGE PRECISION (mAP)

Mean Average Precision (mAP) is a popular metric which is utilized in evaluating object detection models. It calculates the average precision (AP) across various recall thresholds [65], offering a comprehensive assessment and evaluation of the model's performance, especially when dealing with multiple classes or when precision varies with recall [68]. mAP can be estimated as follows:

$$AP = \int_0^1 P(r)dr \tag{5}$$

$$mAP = \frac{1}{k}\sum_{k=i}^{k} AP_i \tag{6}$$

F. INTERSECTION OVER UNION

Intersection over Union (IoU) is a metric used to assess and validate the overlap between predicted and actual bounding boxes [69]. It can be calculated as the ratio of the area of overlap between the two bounding boxes to their combined area [70]:

$$IOU = \frac{area(box(Pred) \cap box(Truth))}{area(box(Pred) \cup box(Truth))} \tag{7}$$

G. CONFUSION MATRIX

The confusion matrix is an invaluable metric in machine learning, providing a detailed visualization of a classifier's performance that transcends traditional accuracy metrics [71]. By analyzing the confusion matrix, users can

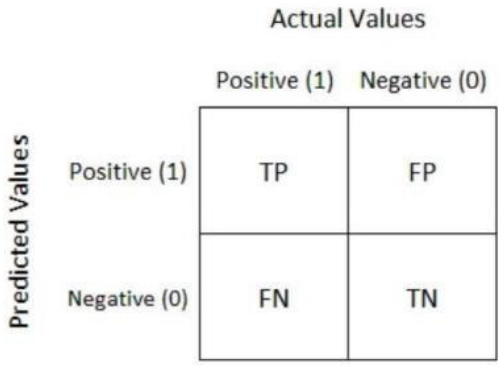


FIGURE 18. Confusion matrix.

TABLE 2. Vision-based datasets for fall detection.

Dataset	Year	Imaging system	Fall type	Scene	Environment
Multiple cameras [72]	2010	RGB-Multiple cameras	falls in different directions	indoor	-
Le2i [73]	2012	RGB multiple cameras	falls in different directions	indoor	-
SDUFall [74]	2014	Kinect Single or Multiple Camera	falls in different directions	Indoor, outdoor	-
UR Fall Detection [75]	2014	Two Kinect Single or Multiple Camera	falling from standing position, falling down sitting on a chai	indoor	different lighting conditions
EDF [76]	2014	Two Kinect Single or Multiple Camera	falls in eight different directions	indoor	-
OCCU [76]	2014	Two Kinect Single or Multiple Camera	falls in eight different directions	indoor	-
FUKinect-Fall [77]	2016	Kinect Single Camera	falls in different directions	indoor	different lighting conditions
Fall Detection [78]	2017	RGB Kinect Single Camera	falls in different directions	indoor	different lighting conditions
UPFall [79]	2019	RGB Two Camera	falls in different directions	indoor	different lighting conditions
CAUCAFall [80]	2022	RGB Single Camera	falls in different directions	indoor	different lighting conditions
CMDFALL [81]	2018	Six Kinect Multiple Camera	falls in eight different directions	indoor	different lighting conditions

gain insights into a classifier's strengths and weaknesses, particularly in terms of sensitivity to specific error types and biases towards certain classes. By understanding the underlying causes of various error's types revealed through the matrix, developers can make informed decisions to enhance model accuracy and robustness, tailoring adjustments to improve either recall or precision as dictated by the application's requirements. Figure 18 illustrates the structure of the confusion matrix.

V. DATASET

In this section, we will present a comprehensive overview of the most widely utilized fall datasets. Falls constitute a significant public health concern, particularly among older adults. These datasets offer a rich repository of information pertaining to the prevalence, risk factors, consequences, and prevention of falls. By analyzing these data, researchers and healthcare professionals can gain a deeper understanding of this critical issue and develop effective interventions.

The datasets under consideration encompass a diverse range of data points, including falls among older adults residing independently in the community and individuals with physical disabilities. These data can be employed to identify specific factors that contribute to falls, such as visual impairment, gait and balance problems, and environmental hazards. This knowledge can then be applied to design targeted interventions aimed at mitigating these risk factors [82]. Additionally, the datasets shed light on the substantial financial burden associated with falls, particularly on the Medicare system [83]. The Fall Data Collection serves as a valuable resource for researchers, healthcare professionals, and policymakers seeking to comprehend the multifaceted nature of falls and develop evidence-based interventions to enhance the quality of life for older adults and individuals with physical disabilities. These datasets typically consist of video recordings or sensor data collected during activities where falls may occur, such as everyday scenarios. There are several datasets related to the detection and analysis of falls in humans that have been used by researchers in the field of deep learning (as presented in Table 2), however, in this review we will focus on the most popular datasets that actually provide a benchmark for fair evaluation and comparison. The most popular datasets used in the field of fall detection with YOLO models can bring different fall scenarios to the experemnts, these datasets including The Pascal Visual Object Classes (VOC 2007) [84], Multiple Cameras Fall Dataset [72], The Microsoft Common Objects in Context (COCO) [85], Large-scale Elderly Falls Detection (LE2I) [86], CAUCAFall [80], and Computer Vision Project [87] datasets. VOC dataset [84] has been extensively utilized in training, testing and evaluating YOLO models for fall detection tasks. The VOC dataset offers a rich collection of images with diverse object classes, including people in various poses and activities. By training YOLO models on VOC dataset, researchers can effectively learn to detect and localize human figures within images, which is crucial for fall detection. Furthermore, the VOC dataset provides a standardized benchmark for evaluating the performance of YOLO models on object detection tasks, enabling researchers to compare different models and identify areas for improvement. The combination of the VOC dataset and YOLO models provides a powerful framework for developing robust and effective fall detection systems.

The utilization of multi-camera in YOLO models for fall detection offers a comprehensive approach to address the challenges associated with real-world fall detection scenarios [72]. Multi-camera datasets play a huge role in developing robust fall detection models that can effectively handle varying camera perspectives and, mitigate the impact of calibration errors [88]. By leveraging the UP-Fall Detection dataset, researchers have successfully utilized optical flow methods to analyze image sequences in fixed time windows,

and extract relevant motion information. These multi-view based approaches have demonstrated superior performance in detecting human falls, achieving high accuracy rates of up to 95.64% compared to state-of-the-art single-view CNN-based methods [89].

COCO [80] dataset, renowned for its rich annotation of instance segmentation, object detection, and captioning tasks, has proven to be a valuable resource for training and evaluating YOLO models for fall detection. COCO's diverse collection of images, featuring a wide range of object categories and complex scenes, provides a robust foundation for learning to detect and localize human figures in challenging environments. By training YOLO models on COCO, researchers can effectively acquire the ability to identify human poses and detect abnormal postures indicative of a fall. Furthermore, COCO's emphasis on instance segmentation allows for precise localization of human body parts, enabling more accurate fall detection and analysis. The combination of COCO's comprehensive annotations and YOLO's efficient object detection capabilities offers a promising approach to developing robust and reliable fall detection systems.

The LE2I Dataset [86] offers a valuable resource for training and evaluating YOLO models for fall detection, particularly in real-world scenarios involving elderly individuals. LE2I provides a diverse collection of video sequences capturing falls from various angles and perspectives, including indoor and outdoor environments. The dataset's focus on elderly individuals is particularly relevant to fall detection applications, as falls are a significant health concern for this population. By training YOLO models on LE2I, researchers can effectively learn to detect and localize elderly individuals within complex scenes and identify abnormal postures indicative of a fall. Furthermore, LE2I's large scale and diversity contribute to the development of models that are robust to variations in lighting conditions, occlusions, and the presence of clutter, enhancing their performance in real-world applications.

The CAUCAFall dataset [80], specifically designed for fall detection in challenging real-world scenarios, offers a valuable resource for training and evaluating YOLO models. CAUCAFall provides a diverse collection of video sequences captured from various camera angles and perspectives, including indoor and outdoor environments. The dataset features a variety of fall types, such as slips, trips, and falls from standing, as well as different clothing styles and backgrounds. By training YOLO models on CAUCAFall, researchers can effectively learn to detect and localize human figures within complex scenes and identify abnormal postures indicative of a fall. Furthermore, CAUCAFall's focus on challenging scenarios, such as occlusions, low-light conditions, and cluttered environments, contributes to the development of models that are robust and reliable in real-world applications.

The Computer Vision Project Dataset [87], while not specifically designed for fall detection, can still be a valuable resource for training and evaluating YOLO models in this domain. If the dataset contains images or videos with human figures in various poses and activities, it can provide a foundation for learning to detect and localize human figures within complex scenes. By carefully selecting relevant data from the Computer Vision Project Dataset and augmenting it with additional fall-related examples, researchers can create a training set that enables YOLO models to effectively identify abnormal postures indicative of a fall. However, it is important to consider the limitations of the dataset and its relevance to fall detection tasks before utilizing it for training. To sum up, we present a comparison in Table 3 between all the above datasets, including the number of images in each dataset, Box boundaries, R, and the final accuracy. All these results were collected from implementing the improved YOLOv8 network.

TABLE 3. Implement the model on various type of datasets.

Dataset	Images	Box(P)	R	mAP%
VOC2007 [84]	4952	0.63	0.549	0.606
multi-camera dataset [72]	1208	0.799	0.743	0.80
COCO2014 [80]	5438	0.773	0.536	0.624
LE2I dataset [86]	3540	0.992	0.981	0.980
CAUCAFall [80]	2317	0.999	1	0.995
Computer Vision Project Dataset [87]	1000	0.851	0.781	0.856

VI. DISCUSSION

The YOLO family of object detection models has consistently sought to balance speed and accuracy, aiming to deliver real-time performance without any potential detection quality. This trade-off has been a recurring theme throughout the evolution of the YOLO family frameworks, with each iteration striving to optimize these competing goals. The original YOLO model prioritized speed as the main objective, enabling real-time processing. However, this prioritization came at the cost of accuracy, particularly for small objects. Subsequent iterations, introduced YOLOv3 model to present enhancements to address these limitations while maintaining real-time capabilities, resulting in improved accuracy. YOLOv3's multi-scale feature extraction network further enhanced performance. In the subsequent versions such as YOLOv4 and YOLOv5, the speed-accuracy trade-off has become more nuanced. Innovations in network structure, data augmentation, and training strategies have led to significant accuracy gains while preserving real-time performance. Since the innovation of YOLOv5, official YOLO models have offered various metrics to cater to specific applications and hardware constraints, providing lightweight models that prioritize speed over accuracy for resource-limited environments.

Researches utilized YOLO for fall detection with two options, including baseline YOLO, and modified YOLO. Table 4 presents recent methods that handle fall detection, including their methodology, the utilized dataset, metrics, mAP results, and their research limitations. Baseline YOLO refer to the original proposed structure without any additional

TABLE 4. Summery of related work on fall detection with YOLO.

Reference	Methodology	Dataset	Metrics	Result	Limitations
Lu and Chu [52]	Deploy YOLOv3 for detection and Camshift for tracking	Simulated fall scenarios, Youtube and Giphy videos	Accuracy	0.9596	Inaccuracies in YOLOv3 and Camshift tracking can cause fall detection false negatives with challenges in complex movements or occlusions.
Long et al. [90]	Leveraged YOLOv3 under daylight conditions	-	Accuracy	0.9200	Reliance on clear visibility may limit effectiveness in poorly lit environments, making detection performance vulnerable to varying lighting conditions
Kamble et al. [91]	Applied YOLO with width-height method to detect falls	UCF-101	Accuracy	0.9316	Limited to width-height criteria which may not met due to positional variations. Furthermore, dynamic or obstructive scenarios may increase false negatives.
Sarıçayır et al. [92]	YOLOv7 and YOLOv8 frameworks	Fallen New Version Dataset	Accuracy	0.8100 and 0.7800; respectively	YOLOv7 may have delays in processing, impacting real-time applications, and YOLOv8's faster processing might sacrifice some accuracy, leading to possible false negatives.
Ke et al. [93]	Tested all YOLOv5 variants, identifying YOLOv5x as the most accurate.	CAUCAFall	Precision	0.8220	YOLOv5x suffers from computational cost speed, which hinders its application in real-time detection.
Kan et al. [94]	Designed CGNS-YOLO with custom modules for precision improvement	URFall, Multicam and Le2i Fall Detection dataset	mAP	0.9130	Reconfiguration may introduce instability in performance due to reliance on the new modules. Furthermore, complexity introduced by multiple modules may hinder interpretability and debugging.
Zheng et al. [95]	Developed FDT-YOLO by enhancing the YOLOv8 framework with new modules for feature reuse.	Le2i , Multicam, Fall detection datasets, and some real-world images obtained from the internet	mAP	0.9620	Suffers from computational resources, potentially hindering speed. Furthermore, the effectiveness of new modules in diverse environments still needs validation.
Luo et al. [96]	Improved YOLOv5s-GCC	URFall, Fall Detection, and Multicam Datasets	mAP	0.9190	limited by generalize well across different datasets, affecting performance in varied scenarios with inconsistencies in feature extraction efficacy.
Zhao et al. [97]	YOLOv7-fall based attention module	URFall, Multicam, Fall Detection datasets, and photos from internet	mAP	0.9500	focus on specific features may overlook other important contextual cues, leading to missed detections. Furthermore, complexity from added modules can slow down inference times in resource-constrained environments.
Sha et al. [98]	YOLOv7-fall based attention module	COCO, URFall, UMAFall, and Youtube videos	mAP	0.9180	compromise detection accuracy in challenging conditions, and effective inference may need careful tuning, which could add to model complexity.
Chen et al. [99]	Enhanced YOLOv5 with an Efficient Channel Attention network	Public dataset	mAP	0.8829	increased model size and complexity, potentially hindering real-time applications. Furthermore, the precision might vary based on the scale of fall events presented.
Chen et al. [100]	Improved YOLOv5s	. URFall	mAP	0.9720	Removing feature layers can lead to loss of critical information regarding smaller objects, affecting overall detection capabilities. Furthermore, the model may require extensive retraining for optimal performance following architecture modification.
Khan et al. [101]	Integrated a focus module in improved YOLOV8S model to optimize feature extraction	Diverse FALL10500	-	-	Increased in computational load, impacting real-time performance. Furthermore, the dependencies on certain features may limit adaptability in diverse scenarios.
Qin et al. [102]	Enhanced YOLOv8S	URFall, Fall Detection Dataset, and internet images	mAP	0.8870	Difficulties in consistent performance across different lighting and environmental conditions.

components, Table 5 shows each YOLO version with it is corresponding network architectural. As for detection part, actually the literature shows that only form YOLOv5 researches deployed YOLO fo fall detection. We present a comparison on baseline YOLO versions in Table 6. This comparison include number of layers, number of trainable

TABLE 5. Key features and architectural evolution of YOLOs.

Algorithm	Year	Architecture
YOLOv1	2016	GoogLeNet-inspired
YOLOv2	2017	Darknet-19, VGG
YOLOv3	2018	Darknet-53
YOLOv4	2020	CSPDDarknet-5
YOLOv5	2020	CSPDDarknet-5
YOLOv6	2021	EfficientRep, CSPDDarknet-5
YOLOv7	2022	ELAN,CSPDDarknet-53
YOLOv8	2023	CSPDDarknet-53
YOLOv9	2024	Generalized ELAN

parameters, and their corresponding accuracy value for fall detection task.

To improve the performance of the baseline network YOLO structure, authors modified the baseline network structure by adding attention layers, removing convolutional layers, Zheng et al. [95] presented the FDT-YOLO algorithm that enhanced fall detection by modifying the YOLOv8 framework, replacing the C2f module with the FasterNet module for better feature reuse and reduced computational complexity. It also integrates a deformable convolution module and triplet attention mechanism, leading to improved detection accuracy and robustness. Zhao et al. [97] introduced YOLOv7-fall, by adopting a novel attention module, SDI, based on Coordinate Attention and Shuffle Attention techniques. Additionally, they used ghost convolutions in the head to reduce parameters and computational complexity. Qin et al. [102] also improved the YOLOv8S model by replacing C2f with C2Dv3 modules in the backbone and adding a DyHead block in the neck layer for unifying multiple attention operations, Table 7 presents a comparison of state-of-the-art method in fall detection.

Due to the rarity and diversity of real-life falls, it is difficult to collect datasets of actual, unexpected falls [79]. Most fall detection datasets are simulated in laboratory settings. There are many limited multimodal datasets with different human activities for falls. Another important factor is that the best results were obtained in environments that had good lighting and few objects in the environment, it was also identified that an area with poor lighting makes it difficult for the estimation model to identify a person [103]. This directly affects the detection model, since, by losing track of the reference points of the person, the algorithm receives incorrect data, consequently, the number of false positives increases [104].

VII. FUTURE WORK OF FALL DETECTION USING YOLO MODELS

The field of fall detection using YOLO models is a rapidly evolving research area with significant potential for improvement. The proposed future research directions can be summarized into three folds:

1) **Dataset Expansion and Diversification:** To enhance the robustness and generalization of YOLO models with different aspect, it is essential to expand and diversify the available datasets. This direction involves collecting data from a wider range of environments, including outdoor settings, different lighting conditions, and various fall types. Furthermore, incorporating datasets with annotations for specific body parts or actions can provide more granular information for fall detection.
2) **Multimodal Approaches:** Combining different YOLO models with other modalities, such as depth sensors or wearable devices, can improve the accuracy and reliability of fall detection. With this integration of data from multiple sources, it is possible to overcome limitations of single-modal approaches and detect falls in more challenging scenarios. For instance, depth sensors can provide information about the 3D structure of the scene, aiding in the detection of falls that may be obscured from a single camera perspective.
3) **Real-time Performance Optimization:** While YOLO models are known for their speed, further steps of optimizations on those models are necessary to achieve real-time performance on resource-constrained devices. These optimizations can be involving techniques such as model compression, quantization, and hardware acceleration. Moreover, exploring efficient inference frameworks and algorithms can help reduce computational overhead and improve latency.

TABLE 6. Fall detection comparison of YOLO models.

Model	layera	parameter	mAP
YOLOv5	214	702,502	0.830
YOLOv6	300	343,111	0.828
YOLOv7	415	37,201,950	0.835
YOLOv8	225	3,011,238	0.86

TABLE 7. Comparison with the state-of-the art methods.

Method	Model	mAP Trainable	Parameters
Zheng et al. [95]	Improved YOLOv8	0.962	9979000
Zhao et al. [97]	Improved YOLOv7	0.95	6017744
Tan [105]	EfficientDet	0.89	-
Qin et al. [102]	Improved YOLOv8	0.887	-
ahlam et al. [106]	YOLOv8 with improvements	0.995	2,155,4366

VIII. CONCLUSION

This paper presents a review on the field of fall detection with different YOLO versions. We first investigate the field on the major fall detection procedure in AI models. Then, we explore different versions of YOLO model, starting from v1 till v8. Next, we dive into each models with it is own structure. It should be noted that starting from YOLOv5, researches explore the evaluation of fall detection with this YOLO version. After that, we describe all metrics that researches evaluated their models performance. Finally, we introduce the major datasets int he field of fall detection, and focus on the most popular datasets. To have more comprehensive and clear understanding of all aspects, we dive into deep discussion of the overall performance of the latest fall detection techniques via different YOLO versions, including

baseline and optimized models, the utilized datasets, metrics, results, and their limitations. To help the readers to explore this field, we sum up this work by presenting three major future research directions.

REFERENCES

[1] B. Luo, "Human fall detection for smart home caring using YOLO networks," *Int. J. Adv. Comput. Sci. Appl.*, vol. 14, no. 4, pp. 59–68, 2023.

[2] R. Igual, C. Medrano, and I. Plaza, "Challenges, issues and trends in fall detection systems," *Biomed. Eng. OnLine*, vol. 12, no. 1, p. 66, 2013.

[3] D. L. Shung and J. J. Y. Sung, "Challenges of developing artificial intelligence-assisted tools for clinical medicine," *J. Gastroenterol. Hepatol.*, vol. 36, no. 2, pp. 295–298, Feb. 2021.

[4] I. H. Sarker, "Deep learning: A comprehensive overview on techniques, taxonomy, applications and research directions," *Social Netw. Comput. Sci.*, vol. 2, no. 6, p. 420, Nov. 2021.

[5] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 779–788.

[6] M. I. Jordan and T. M. Mitchell, "Machine learning: Trends, perspectives, and prospects," *Science*, vol. 349, no. 6245, pp. 255–260, Jul. 2015.

[7] S. Chaudhuri, H. Thompson, and G. Demiris, "Fall detection devices and their use with older adults: A systematic review," *J. Geriatric Phys. Therapy*, vol. 37, no. 4, pp. 178–196, 2014.

[8] X. Wang, J. Ellul, and G. Azzopardi, "Elderly fall detection systems: A literature survey," *Frontiers Robot. AI*, vol. 7, p. 71, Jun. 2020.

[9] V. Jakkula, "Tutorial on Support Vector Machine (SVM)," Washington State Univ., Washington, DC, USA, 2011.

[10] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, pp. 5–32, Oct. 2001.

[11] R. M. Cichy and D. Kaiser, "Deep neural networks as scientific models," *Trends Cognit. Sci.*, vol. 23, no. 4, pp. 305–317, Apr. 2019.

[12] S. Suthaharan, "Support vector machine," in *Machine Learning Models and Algorithms for Big Data Classification: Thinking With Examples for Effective Learning*, vol. 36. Boston, MA, USA: Springer, 2016, pp. 207–235.

[13] O. Kramer and O. Kramer, "K-nearest neighbors," in *Dimensionality Reduction With Unsupervised Nearest Neighbors*, vol. 51. Berlin, Germany: Springer, 2013, pp. 13–23.

[14] S. J. Rigatti, "Random forest," *J. Insurance Med.*, vol. 47, no. 1, pp. 31–39, 2017.

[15] C. Rougier, J. Meunier, A. St-Arnaud, and J. Rousseau, "Robust video surveillance for fall detection based on human shape deformation," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 21, no. 5, pp. 611–622, May 2011.

[16] L. Anishchenko, A. Zhuravlev, and M. Chizh, "Fall detection using multiple bioradars and convolutional neural networks," *Sensors*, vol. 19, no. 24, p. 5569, Dec. 2019.

[17] B. Mahesh, "Machine learning algorithms—A review," *Int. J. Sci. Res.*, vol. 9, no. 1, pp. 381–386, Jan. 2020.

[18] I. Charfi, J. Miteran, J. Dubois, M. Atri, and R. Tourki, "Definition and performance evaluation of a robust SVM based fall detection solution," in *Proc. 8th Int. Conf. Signal Image Technol. Internet Based Syst.*, Nov. 2012, pp. 218–224.

[19] K. Taunk, S. De, S. Verma, and A. Swetapadma, "A brief review of nearest neighbor algorithm for learning and classification," in *Proc. Int. Conf. Intell. Comput. Control Syst. (ICCS)*, May 2019, pp. 1255–1260.

[20] C.-L. Liu, C.-H. Lee, and P.-M. Lin, "A fall detection system using k-nearest neighbor classifier," *Expert Syst. Appl.*, vol. 37, no. 10, pp. 7174–7181, Oct. 2010.

[21] G. Biau and E. Scornet, "A random forest guided tour," *Test*, vol. 25, no. 2, pp. 197–227, Jun. 2016.

[22] C. Liu, Z. Gu, and J. Wang, "A hybrid intrusion detection system based on scalable K-means+ random forest and deep learning," *IEEE Access*, vol. 9, pp. 75729–75740, 2021.

[23] N. O'Mahony, S. Campbell, A. Carvalho, S. Harapanahalli, G. V. Hernandez, L. Krpalkova, D. Riordan, and J. Walsh, "Deep learning vs. traditional computer vision," in *Proc. Comput. Vis. Conf. (CVC)*, vol. 1. Cham, Switzerland: Springer, 2020, pp. 128–144.

[24] G. L. Santos, P. T. Endo, K. H. D. C. Monteiro, E. D. S. Rocha, I. Silva, and T. Lynn, "Accelerometer-based human fall detection using convolutional neural networks," *Sensors*, vol. 19, no. 7, p. 1644, Apr. 2019.

[25] M. Yu, L. Gong, and S. Kollias, "Computer vision based fall detection by a convolutional neural network," in *Proc. 19th ACM Int. Conf. Multimodal Interact.*, Nov. 2017, pp. 416–420.

[26] S. K. Yadav, A. Luthra, K. Tiwari, H. M. Pandey, and S. A. Akbar, "ARFDNet: An efficient activity recognition & fall detection system using latent feature pooling," *Knowl.-Based Syst.*, vol. 239, Mar. 2022, Art. no. 107948.

[27] D. Maji, S. Nagori, M. Mathew, and D. Poddar, "YOLO-pose: Enhancing YOLO for multi person pose estimation using object keypoint similarity loss," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2022, pp. 2636–2645.

[28] M. Carranza-García, J. Torres-Mateo, P. Lara-Benítez, and J. García-Gutiérrez, "On the performance of one-stage and two-stage object detectors in autonomous vehicles using camera data," *Remote Sens.*, vol. 13, no. 1, p. 89, Dec. 2020.

[29] A. Benjumea, I. Teeti, F. Cuzzolin, and A. Bradley, "YOLO-Z: Improving small object detection in YOLOv5 for autonomous vehicles," 2021, *arXiv:2112.11798*.

[30] M. A. Arroyo, M. T. I. Ziad, H. Kobayashi, J. Yang, and S. Sethumadhavan, "YOLO: Frequently resetting cyber-physical systems for security," *Proc. SPIE*, vol. 11009, pp. 166–183, Jan. 2019.

[31] M. Ş. Gündüz and G. Işık, "A new YOLO-based method for real-time crowd detection from video and performance analysis of YOLO models," *J. Real-Time Image Process.*, vol. 20, no. 1, p. 5, Feb. 2023.

[32] D. Theckedath and R. R. Sedamkar, "Detecting affect states using VGG16, ResNet50 and SE-ResNet50 networks," *Social Netw. Comput. Sci.*, vol. 1, no. 2, p. 79, Mar. 2020.

[33] B. Koonce and B. Koonce, "ResNet 50," in *Convolutional Neural Networks With Swift for Tensorflow: Image Recognition and Dataset Categorization*. Berkeley, CA, USA: Apress, 2021, pp. 63–72.

[34] M. Mahasin and I. A. Dewi, "Comparison of CSPDarkNet53, CSPResNeXt-50, and EfficientNet-B0 backbones on YOLO V4 as object detector," *Int. J. Eng., Sci. Inf. Technol.*, vol. 2, no. 3, pp. 64–72, Sep. 2022.

[35] K. Weng, X. Chu, X. Xu, J. Huang, and X. Wei, "EfficientRep: An efficient repvgg-style ConvNets with hardware-aware neural network design," 2023, *arXiv:2302.00386*.

[36] K. He, X. Zhang, S. Ren, and J. Sun, "Spatial pyramid pooling in deep convolutional networks for visual recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 37, no. 9, pp. 1904–1916, Sep. 2015.

[37] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 936–944.

[38] G. Ghiasi, T.-Y. Lin, and Q. V. Le, "NAS-FPN: Learning scalable feature pyramid architecture for object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 7029–7038.

[39] L. Bai, H. Song, T. Feng, T. Fu, Q. Yu, and J. Yang, "Revisiting class-incremental object detection: An efficient approach via intrinsic characteristics alignment and task decoupling," *Expert Syst. Appl.*, vol. 257, Dec. 2024, Art. no. 125057.

[40] J. Li, J. Zhang, W. Ge, and X. Liu, "Multi-scale methodology for complex systems," *Chem. Eng. Sci.*, vol. 59, nos. 8–9, pp. 1687–1700, Apr. 2004.

[41] T. Kong, F. Sun, H. Liu, Y. Jiang, L. Li, and J. Shi, "FoveaBox: Beyound anchor-based object detection," *IEEE Trans. Image Process.*, vol. 29, pp. 7389–7398, 2020.

[42] C. Chen, Z. Zheng, T. Xu, S. Guo, S. Feng, W. Yao, and Y. Lan, "YOLO-based UAV technology: A review of the research and its applications," *Drones*, vol. 7, no. 3, p. 190, Mar. 2023.

[43] Z.-Q. Zhao, P. Zheng, S.-T. Xu, and X. Wu, "Object detection with deep learning: A review," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 11, pp. 3212–3232, Nov. 2019.

[44] W. Fang, L. Wang, and P. Ren, "Tinier-YOLO: A real-time object detection method for constrained environments," *IEEE Access*, vol. 8, pp. 1935–1944, 2020.

[45] P. Jiang, D. Ergu, F. Liu, Y. Cai, and B. Ma, "A review of YOLO algorithm developments," *Proc. Comput. Sci.*, vol. 199, pp. 1066–1073, Jan. 2022.

[46] S.-H. Han, K. W. Kim, S. Kim, and Y. C. Youn, "Artificial neural network: Understanding the basic concepts without mathematics," *Dementia Neurocognitive Disorders*, vol. 17, no. 3, p. 83, 2018.

[47] A. Nazir and Mohd. A. Wani, "You only look once–object detection models: A review," in *Proc. 10th Int. Conf. Comput. Sustain. Global Develop. (INDIACom)*, Mar. 2023, pp. 1088–1095.
[48] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," 2018, *arXiv:1804.02767*.
[49] P. Adarsh, P. Rathi, and M. Kumar, "YOLO v3-tiny: Object detection and recognition using one stage improved model," in *Proc. 6th Int. Conf. Adv. Comput. Commun. Syst. (ICACCS)*, Mar. 2020, pp. 687–694.
[50] K. Wang and M. Liu, "YOLOv3-MT: A YOLOv3 using multi-target tracking for vehicle visual detection," *Int. J. Speech Technol.*, vol. 52, no. 2, pp. 2070–2091, Jan. 2022.
[51] X. Zhang, X. Dong, Q. Wei, and K. Zhou, "Real-time object detection algorithm based on improved YOLOv3," *J. Electron. Imag.*, vol. 28, no. 5, Oct. 2019, Art. no. 053022.
[52] K.-L. Lu and E. T.-H. Chu, "An image-based fall detection system for the elderly," *Appl. Sci.*, vol. 8, no. 10, p. 1995, Oct. 2018.
[53] A. Bochkovskiy, C.-Y. Wang, and H.-Y. Mark Liao, "YOLOv4: Optimal speed and accuracy of object detection," 2020, *arXiv:2004.10934*.
[54] G. Jocher. (2020). *Overview of Model Structure About YOLOv5*. [Online]. Available: https://github.com/ultralytics/yolov5
[55] C. Li, L. Li, H. Jiang, K. Weng, Y. Geng, L. Li, Z. Ke, Q. Li, M. Cheng, W. Nie, Y. Li, B. Zhang, Y. Liang, L. Zhou, X. Xu, X. Chu, X. Wei, and X. Wei, "YOLOv6: A single-stage object detection framework for industrial applications," 2022, *arXiv:2209.02976*.
[56] X. Ding, H. Chen, X. Zhang, K. Huang, J. Han, and G. Ding, "Re-parameterizing your optimizers rather than architectures," 2022, *arXiv:2205.15242*.
[57] C. Zhao, J. H. Hsiao, and A. B. Chan, "Gradient-based instance-specific visual explanations for object specification and object discrimination," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 9, pp. 5967–5985, Sep. 2024.
[58] C.-Y. Wang, A. Bochkovskiy, and H.-Y.-M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 7464–7475.
[59] F. J. Solawetz. (2023). *What is YOLOv8? The Ultimate Guide*. [Online]. Available: https://blog.roboflow.com/whats-new-in-yolov8
[60] S. N. Moutsis, K. A. Tsintotas, I. Kansizoglou, S. An, Y. Aloimonos, and A. Gasteratos, "Fall detection paradigm for embedded devices based on YOLOv8," in *Proc. IEEE Int. Conf. Imag. Syst. Techn. (IST)*, Oct. 2023, pp. 1–6.
[61] T. Diwan, G. Anirudh, and J. V. Tembhurne, "Object detection using YOLO: Challenges, architectural successors, datasets and applications," *Multimedia Tools Appl.*, vol. 82, no. 6, pp. 9243–9275, Mar. 2023.
[62] G. Astlin Pereira, "Fall detection for industrial setups using YOLOv8 variants," 2024, *arXiv:2408.04605*.
[63] J. Terven and D. Cordova-Esparza, "A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS," 2023, *arXiv:2304.00501*.
[64] G. Jocher, A. Chaurasia, and J. Qiu. (Jan. 2023). *Ultralytics YOLO*. [Online]. Available: https://github.com/ultralytics/ultralytics
[65] M. M. A. Parambil, L. Ali, F. Alnajjar, and M. Gochoo, "Smart classroom: A deep learning approach towards attention assessment through class behavior detection," in *Proc. Adv. Sci. Eng. Technol. Int. Conferences (ASET)*, Feb. 2022, pp. 1–6.
[66] M. Hossin and M. N. Sulaiman, "A review on evaluation metrics for data classification evaluations," *Int. J. Data Mining Knowl. Manage. Process*, vol. 5, no. 2, p. 1, 2015.
[67] S. Varma and M. Sreeraj, "Object detection and classification in surveillance system," in *Proc. IEEE Recent Adv. Intell. Comput. Syst. (RAICS)*, Dec. 2013, pp. 299–303.
[68] R. Padilla, S. L. Netto, and E. A. B. da Silva, "A survey on performance metrics for object-detection algorithms," in *Proc. Int. Conf. Syst., Signals Image Process. (IWSSIP)*, Jul. 2020, pp. 237–242.
[69] J. Yu, Y. Jiang, Z. Wang, Z. Cao, and T. Huang, "UnitBox: An advanced object detection network," in *Proc. 24th ACM Int. Conf. Multimedia*, Oct. 2016, pp. 516–520.
[70] H. Rezatofighi, N. Tsoi, J. Gwak, A. Sadeghian, I. Reid, and S. Savarese, "Generalized intersection over union: A metric and a loss for bounding box regression," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 658–666.
[71] C. S. Hong and T. G. Oh, "TPR-TNR plot for confusion matrix," *Commun. Stat. Appl. Methods*, vol. 28, no. 2, pp. 161–169, Mar. 2021.
[72] E. Auvinet, C. Rougier, J. Meunier, A. St-Arnaud, and J. Rousseau, "Multiple cameras fall dataset," DIRO-Université de Montréal, Montreal, QC, Canada, Tech. Rep. 1350, Jul. 2010.
[73] I. Charfi, J. Miteran, J. Dubois, M. Atri, and R. Tourki, "Optimized spatio-temporal descriptors for real-time fall detection: Comparison of support vector machine and AdaBoost-based classification," *J. Electron. Imag.*, vol. 22, no. 4, Jul. 2013, Art. no. 041106.
[74] X. Ma, H. Wang, B. Xue, M. Zhou, B. Ji, and Y. Li, "Depth-based human fall detection via shape features and improved extreme learning machine," *IEEE J. Biomed. Health Informat.*, vol. 18, no. 6, pp. 1915–1922, Nov. 2014.
[75] B. Kwolek and M. Kepski, "Human fall detection on embedded platform using depth maps and wireless accelerometer," *Comput. Methods Programs Biomed.*, vol. 117, no. 3, pp. 489–501, Dec. 2014.
[76] Z. Zhang, C. Conly, and V. Athitsos, "Evaluating depth-based computer vision methods for fall detection under occlusions," in *Proc. 10th Int. Symp. Adv. Vis. Comput.*, Las Vegas, NV, USA. Cham, Switzerland: Springer, 2014, pp. 196–207.
[77] M. Aslan, Y. Akbulut, A. Şengür, and M. C. Ince, "Skeleton based efficient fall detection," *J. Fac. Eng. Archit. Gazi Univ.*, vol. 32, no. 4, pp. 1025–1034, 2017.
[78] K. Adhikari, H. Bouchachia, and H. Nait-Charif, "Activity recognition for indoor fall detection using convolutional neural network," in *Proc. 15th IAPR Int. Conf. Mach. Vis. Appl. (MVA)*, May 2017, pp. 81–84.
[79] L. Martínez-Villaseñor, H. Ponce, J. Brieva, E. Moya-Albor, J. Núñez-Martínez, and C. Peñafort-Asturiano, "UP-fall detection dataset: A multimodal approach," *Sensors*, vol. 19, no. 9, p. 1988, Apr. 2019.
[80] J. C. E. Guerrero, E. M. España, M. M. Añasco, and J. E. P. Lopera, "Dataset for human fall recognition in an uncontrolled environment," *Data Brief*, vol. 45, Dec. 2022, Art. no. 108610.
[81] T.-H. Tran, T.-L. Le, D.-T. Pham, V.-N. Hoang, V.-M. Khong, Q.-T. Tran, T.-S. Nguyen, and C. Pham, "A multi-modal multi-view dataset for human fall analysis and preliminary investigation on modality," in *Proc. 24th Int. Conf. Pattern Recognit. (ICPR)*, Aug. 2018, pp. 1947–1952.
[82] M. D. Solbach and J. K. Tsotsos, "Vision-based fallen person detection for the elderly," in *Proc. IEEE Int. Conf. Comput. Vis. Workshops*, Oct. 2017, pp. 1433–1442.
[83] F. Li, E. Eckstrom, P. Harmer, K. Fitzgerald, J. Voit, and K. A. Cameron, "Exercise and fall prevention: Narrowing the research-to-practice gap and enhancing integration of clinical and community practice," *J. Amer. Geriatrics Soc.*, vol. 64, no. 2, pp. 425–431, Feb. 2016.
[84] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, "The PASCAL visual object classes (VOC) challenge," *Int. J. Comput. Vis.*, vol. 88, no. 2, pp. 303–338, Jun. 2010.
[85] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *Proc. 13th Eur. Conf. Comput. Vis.*, Zurich, Switzerland. Cham, Switzerland: Springer, 2014, pp. 740–755.
[86] I. Charfi, J. Miteran, J. Dubois, M. Atri, and R. Tourki, "Optimised spatio-temporal descriptors for real-time fall detection: Comparison of SVM and AdaBoost based classification," *J. Electron. Imag.*, vol. 22, no. 14, p. 17, 2013.
[87] Roboflow Universe Projects. (Mar. 2023). *Fall Detection Dataset*. [Online]. Available: https://universe.roboflow.com/roboflow-universe-projects/fall-detection-ca3o8
[88] F. Abedi, Q. Liu, and Y. Yang, "Multi-view high dynamic range reconstruction via gain estimation," in *Proc. IEEE Vis. Commun. Image Process. (VCIP)*, Dec. 2019, pp. 1–4.
[89] R. Espinosa, H. Ponce, S. Gutiérrez, L. Martínez-Villaseñor, J. Brieva, and E. Moya-Albor, "A vision-based approach for fall detection using multiple cameras and convolutional neural networks: A case study using the UP-fall detection dataset," *Comput. Biol. Med.*, vol. 115, Dec. 2019, Art. no. 103520.
[90] K. Z. Long, H. Haron, M. Ibrahim, and Z. D. Eri, "An image-based fall detection system using you only look once (YOLO) algorithm to monitor elders' fall events," in *Proc. Knowl. Manage. Int. Conf. (KMICe)*, vol. 1, 2021, pp. 50–55.
[91] K. P. Kamble, S. S. Sontakke, H. Donadkar, R. Poshattiwar, and A. Ananth, "Fall alert: A novel approach to detect fall using base as a YOLO object detection," in *Advanced Machine Learning Technologies and Applications*. Cham, Switzerland: Springer, 2021, pp. 15–24.

[92] B. Sarıçayır, E. Alican, and C. Özcan, "Deep learning-based human detection for fall injuries," *Current Trends Comput.*, vol. 1, no. 2, pp. 83–92, 2023.
[93] Y. Ke, Y. Yao, Z. Xie, H. Xie, H. Lin, and C. Dong, "Empowering intelligent home safety: Indoor family fall detection with YOLOv5," in *Proc. IEEE Int. Conf. Dependable, Autonomic Secure Comput., Int. Conf. Pervasive Intell. Comput., Int. Conf. Cloud Big Data Comput., Int. Conf. Cyber Sci. Technol. Congr. (DASC/PiCom/CBDCom/CyberSciTech)*, Nov. 2023, pp. 0942–0949.
[94] X. Kan, S. Zhu, Y. Zhang, and C. Qian, "A lightweight human fall detection network," *Sensors*, vol. 23, no. 22, p. 9069, Nov. 2023.
[95] X. Zheng, J. Cao, C. Wang, and P. Ma, "A high-precision human fall detection model based on FasterNet and deformable convolution," *Electronics*, vol. 13, no. 14, p. 2798, Jul. 2024.
[96] Z. Luo, S. Jia, H. Niu, Y. Zhao, X. Zeng, and G. Dong, "Elderly fall detection algorithm based on improved YOLOv5s," *Inf. Technol. Control*, vol. 53, no. 2, pp. 601–618, Jun. 2024.
[97] D. Zhao, T. Song, J. Gao, D. Li, and Y. Niu, "YOLO-fall: A novel convolutional neural network model for fall detection in open spaces," *IEEE Access*, vol. 12, pp. 26137–26149, 2024.
[98] Y. Sha, X. Zhai, J. Li, W. Meng, H. H. Y. Tong, and K. Li, "A novel lightweight deep learning fall detection system based on global-local attention and channel feature augmentation," *Interdiscipl. Nursing Res.*, vol. 2, no. 2, pp. 68–75, Jul. 2023.
[99] H. Chen, W. Gu, Q. Zhang, X. Li, and X. Jiang, "Integrating attention mechanism and multi-scale feature extraction for fall detection," *Heliyon*, vol. 10, no. 10, May 2024, Art. no. e31614.
[100] T. Chen, Z. Ding, and B. Li, "Elderly fall detection based on improved YOLOv5s network," *IEEE Access*, vol. 10, pp. 91273–91282, 2022.
[101] H. Khan, I. Ullah, M. Shabaz, M. F. Omer, M. T. Usman, M. S. Guellil, and J. Koo, "Visionary vigilance: Optimized YOLOV8 for fallen person detection with large-scale benchmark dataset," *Image Vis. Comput.*, vol. 149, Sep. 2024, Art. no. 105195.
[102] Y. Qin, W. Miao, and C. Qian, "A high-precision fall detection model based on dynamic convolution in complex scenes," *Electronics*, vol. 13, no. 6, p. 1141, Mar. 2024.
[103] M. Elgendi, F. Picon, N. Magnenat-Thalmann, and D. Abbott, "Arm movement speed assessment via a Kinect camera: A preliminary study in healthy subjects," *Biomed. Eng. OnLine*, vol. 13, no. 1, p. 88, 2014.
[104] C. Park, "A wrist-type fall detector with statistical classifier for the elderly care," *KSII Trans. Internet Inf. Syst.*, vol. 5, no. 10, pp. 1751–1768, Oct. 2011.
[105] M. Tan, R. Pang, and Q. V. Le, "EfficientDet: Scalable and efficient object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 10778–10787.
[106] A. R. Khekan, H. S. Aghdasi, and P. Salehpoor, "Fast and high-precision human fall detection using improved YOLOv8 model," *IEEE Access*, early access, Sep. 30, 2024, doi: 10.1109/ACCESS.2024.3470319.



AHLAM R. KHEKAN was born in Baghdad, Iraq. She received the B.Sc. and M.Sc. degrees in computer science from Al-Mustansirya University, in 2000 and 2014, respectively. She is currently pursuing the Ph.D. degree in computer engineering with the University of Tabriz, Tabriz. She is with the BioMedical Informatics College, University of Information Technology and Communications, Baghdad, Iraq. Her research interests include computer science, artificial intelligence, pattern recognition, and bioinformatics.



HADI S. AGHDASI received the B.S. degree in computer engineering from the Sadjad University of Technology, Mashhad, Iran, in 2006, and the M.S. and Ph.D. degrees in computer engineering from Shahid Beheshti University, Tehran, Iran, in 2008 and 2013, respectively. He is currently an Associate Professor with the Computer Engineering Department, University of Tabriz, Tabriz, Iran, where he is also the Director of the Humanoid Robots and Cognitive Technology (HRCT) Laboratory and Wireless-Ad Hoc and Sensor Networks (WASL) Laboratory. His current research interests include humanoid robots and intelligent methods in surveillance systems and cognitive technology. Also, he works on wireless traditional and visual sensor networks (routing, clustering, coverage, and visual information transmission).



PEDRAM SALEHPOUR (Member, IEEE) received the B.S. and M.Sc. degrees in computer science and the Ph.D. degree in electrical engineering from the University of Tabriz, Tabriz, Iran, in 2007, 2009, and 2015, respectively. He is currently an Associate Professor with the Department of Computer Engineering, Faculty of Electrical and Computer Engineering, University of Tabriz. His research interests include distributed computing, machine learning, image processing, and deep learning.